

Amérique du Nord – 2025 – sujet1 - Correction

Exercice 3 (8 points)

Partie A – Base de données

1. Type de l'attribut annee

L'attribut annee est de type INTEGER (ou INT), car il représente une année de naissance.

2. Contrainte de domaine sur annee

Une contrainte pertinente consiste à imposer que l'année soit comprise entre une valeur minimale (par exemple 2005) et l'année courante, afin d'éviter des dates incohérentes.

3. Contrainte de référence

L'attribut num_parent de la table enfant suit une contrainte de référence : il référence l'attribut tel de la table parent.

4. Clef primaire de la table parent

La clef primaire de la table parent peut être l'attribut tel, car il identifie de manière unique chaque parent et est utilisé comme référence.

5. Erreur dans la requête UPDATE

La requête lève une erreur car le numéro de téléphone est une clef primaire référencée par la table enfant. La modification viole temporairement la contrainte d'intégrité référentielle.

6. Suite correcte de commandes SQL

```
INSERT INTO parent VALUES ('Bauges', 33619782812, 73340);
```

```
UPDATE enfant  
SET num_parent = 33619782812  
WHERE num_parent = 33600782812;
```

```
DELETE FROM parent  
WHERE tel = 33600782812;
```

7. Résultat de la requête SQL

La requête renvoie les prénoms des enfants nés avant 2014, triés par année croissante :
Hawa, Kian, Adrien, Nakamura.

8. Enfants d'un parent donné

```
SELECT prenom
FROM enfant
WHERE num_parent = 3619861122
ORDER BY prenom;
```

9. Enfants dont le parent habite en 38520

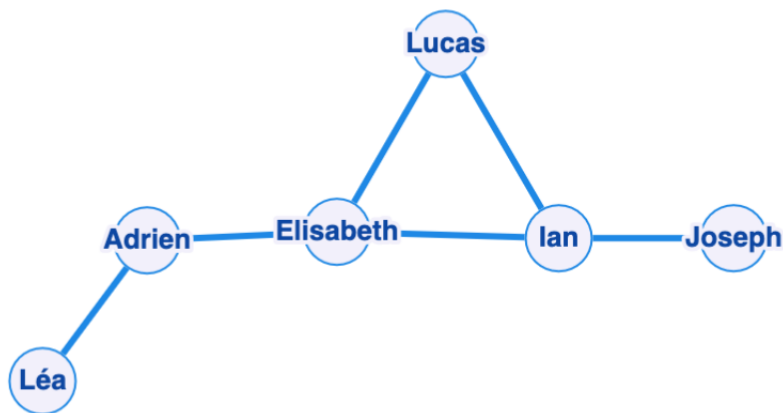
```
SELECT enfant.id, enfant.prenom
FROM enfant
JOIN parent ON enfant.num_parent = parent.tel
WHERE parent.codep = 38520;
```

Partie B – Graphes et algorithmique

10. Graphe non orienté

Une mésentente est réciproque : si A ne s'entend pas avec B, alors B ne s'entend pas avec A. Un graphe non orienté est donc suffisant.

11. Graphe g2



👉 *Commentaire* : Le graphe g2 relie Adrien à Elisabeth et Lea, Elisabeth à Adrien, Ian et Luca, Ian à Elisabeth, Joseph et Luca, Joseph à Ian, Lea à Adrien, et Luca à Elisabeth et Ian.

12. Fonction degre

```
def degre(g, s):  
    return len(g[s])
```

13. Fonction sommets_tries

```
def sommets_tries(g):  
    sommets = list(g.keys())  
    for i in range(len(sommets)):  
        for j in range(i + 1, len(sommets)):  
            if degre(g, sommets[j]) > degre(g, sommets[i]):  
                sommets[i], sommets[j] = sommets[j], sommets[i]  
    return sommets
```

14. Tri et complexité

Le tri utilisé est un tri par insertion.

Son coût dans le pire des cas est quadratique : $O(n^2)$.

15. Coloration du graphe g1 avec 3 couleurs

Une coloration possible est par exemple :

Elise : 0, Octavie : 1, Pierre : 0, Raphael : 1, Virgile : 2, Sixtine : 0.

16. Procédure de coloration

```
def colorer_graphe(g, dc):  
    # Pré-condition : les clés de dc sont les sommets  
    # de g, et les valeurs de dc sont toutes à -1  
    for s in dc:  
        couleur = plus_petite_couleur_hors_voisins(g, dc, s)  
        dc[s] = couleur
```

17. code welsh_powel

```
def welsh_powell(g):  
    # initialisation à -1 pour tous les sommets dans le  
    dictionnaire dc  
    dc = {}  
    for s in g:  
        dc[s] = -1  
        # coloration en suivant l'approche de Welsh-Powell  
        for s in sommets_tries(g):  
            couleur = plus_petite_couleur_hors_voisins(g, dc, s)  
            dc[s] = couleur  
    return dc
```
